Modeling and Control of a Double Pendulum: Swing-Up, Stabilization and Disturbance Rejection

Konstantinos Spinos

***Abstract*—Trajectory optimization (TO) is a technique or a subprocess of a general purpose - motion planning. In practice, motion planning comes with many problems including computational complexity, uncertainty, and physical constraints. To address these issues, a common approach is to use trajectory optimization methods. TO focuses on finding the best path that minimizes a given cost function (most likely time or energy) while respecting the physical constraints of the system. In this paper, a swing-up trajectory of a double pendulum is computed with Hermite-Simpson direct collocation and the IPOPT solver. The optimized trajectory is then tracked with a Time-Varying Linear Quadratic Regulator (TVLQR), and an infinite-horizon LQR with integral action holds the pendulum at the top vertical position. The complete controller respects the actuator limits and is tested against measurement noise and against disturbance torques (short pushes and a sustained force) applied to the system.**

I. Introduction

The paper concerns the modeling, trajectory optimization, and control of a double pendulum system, as seen in Figure 1. More precisely, the trajectory optimization is solved with the Hermite-Simpson method, the optimized swing-up is tracked via TVLQR, and the pendulum is stabilized at the top vertical position, despite disturbances, by an infinite-horizon LQR with integral action.

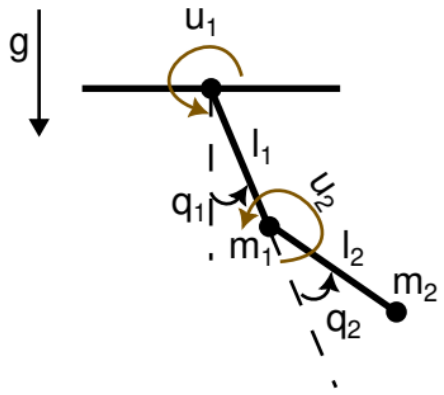


Fig. 1. Double Pendulum System

II. Modeling

The system consists of two point masses m_1, m_2 connected by massless rods of length l_1, l_2 respectively. Both joints exert torque (u_1 and u_2 respectively). The angle q_1 is measured from the downward vertical, and q_2 is the angle of the second rod relative to the first one, so $q_1 = q_2 = 0$ is the hanging position and $q_1 = \pi, q_2 = 0$ is the upright position. The system has four continuous state variables: two joint positions (q_1, q_2) and two joint velocities $(\dot{q}_1, \dot{q}_2)$. The dynamics of the system along with the state and the control input is as follows:

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \ddot{q}_1 \\ \ddot{q}_2 \end{bmatrix} \in \mathbb{R}^4$$

$$\mathbf{x} = \begin{bmatrix} q_1 \\ q_2 \\ \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} \in \mathbb{R}^4 \quad \mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \in \mathbb{R}^2$$

By defining $\mathbf{q} = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}$ we can derive the following forward dynamics equation:

$$\ddot{\mathbf{q}} = \mathbf{M}^{-1}(\mathbf{u} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}))$$

where

$$\mathbf{M} = \begin{bmatrix} l_1^2 m_1 + l_2^2 m_2 + l_1^2 m_2 + 2 l_1 m_2 l_2 \cos q_2 & l_2^2 m_2 + l_1 m_2 l_2 \cos q_2 \\ l_2^2 m_2 + l_1 m_2 l_2 \cos q_2 & l_2^2 m_2 \end{bmatrix}$$

$$\mathbf{C} = \begin{bmatrix} -2\dot{q}_2 l_1 m_2 l_2 \sin q_2 & -\dot{q}_2 l_1 m_2 l_2 \sin q_2 \\ \dot{q}_1 l_1 m_2 l_2 \sin q_2 & 0 \end{bmatrix}$$

$$\mathbf{G} = \begin{bmatrix} -g m_1 l_1 \sin q_1 - g m_2 (l_1 \sin q_1 + l_2 \sin(q_1 + q_2)) \\ -g m_2 l_2 \sin(q_1 + q_2) \end{bmatrix}$$

and $g = 9.81\, m/s^2$, $m_1 = m_2 = 1\, kg$, $l_1 = l_2 = 0.5\, m$. The dimension of each matrix is: $\mathbf{M} \in \mathbb{R}^{2\times2}, \mathbf{C} \in \mathbb{R}^{2\times2}, \mathbf{G} \in \mathbb{R}^{2\times1}$.

A simple visualization of the system can be seen below (initial state $x_{init} = [0.3 \quad 0.9 \quad 0 \quad 0]^T$). The system has been discretized by the Runge-Kutta 4th Order integrator with a time step $dt = 0.05\, s$, assuming that the control input is constant (zero-order hold) during each step.

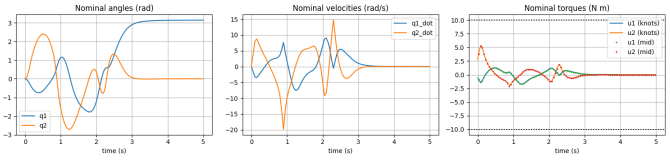


Fig. 2. System Visualization

III. Trajectory Optimization

In this section, our aim is to build an optimized trajectory for the pendulum system. It's initial and final state will respectfully be:

$$\mathbf{x}_0 = \begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix}^T \qquad \mathbf{x}_f = \begin{bmatrix} \pi & 0 & 0 & 0 \end{bmatrix}^T$$

The double pendulum will thus begin at the bottom vertical, where the only force applied is gravity, while the goal is to enforce a swing-up behavior and move the pendulum system at the top vertical point within $T = 5\,s$. Physical limitations of the actuators require the control inputs to be in bounds: $-10 \leq \mathbf{u} \leq 10\ \mathrm{N\,m}$. In addition to the physical constraints of the system, we need to take into account the boundary conditions as well as the constraints applied by the Hermite-Simpson method. The horizon is divided into $K = 100$ intervals of length $h = dt = 0.05\,s$ (so there are $K + 1 = 101$ knot points). This method imports the dynamic constraints above:

$$\mathbf{x}_{k+1} - \mathbf{x}_k = \frac{1}{6} h (f_k + 4 f_{k+\frac{1}{2}} + f_{k+1})$$
$$\mathbf{x}_{k+\frac{1}{2}} = \frac{1}{2}(\mathbf{x}_{k+1} + \mathbf{x}_k) + \frac{h}{8}(f_k - f_{k+1})$$

We have two options:

1) Optimize over the variables $\mathbf{x}, \mathbf{u}$ solely, and merge the constraints into one
2) Add mid points $\mathbf{x}_{k+\frac{1}{2}}, \mathbf{u}_{k+\frac{1}{2}}$ as optimization variables and confront separately the two constraints

The first option provides faster evaluation of the cyipopt problem but is more unstable and vulnerable to noise. We choose the second option for a more robust approach. As a result, the optimization variable $\mathbf{z}$ is as seen below.

$$\mathbf{z} = \begin{bmatrix} \mathbf{x} \\ \mathbf{u} \\ \mathbf{x}_m \\ \mathbf{u}_m \end{bmatrix} = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_K \\ u_0 \\ u_1 \\ \vdots \\ u_K \\ x_{\frac{1}{2}} \\ \vdots \\ x_{K-\frac{1}{2}} \\ u_{\frac{1}{2}} \\ \vdots \\ u_{K-\frac{1}{2}} \end{bmatrix} \in \mathbb{R}^{1206}$$

$\mathbf{x} \in \mathbb{R}^{(K+1)\times N}, \mathbf{u} \in \mathbb{R}^{(K+1)\times M}, \mathbf{x}_m \in \mathbb{R}^{K\times N}, \mathbf{u}_m \in \mathbb{R}^{K\times M}$ and $N = 4$ (state size), $M = 2$ (control size), $K = 100$ (intervals). The problem has $2N + 2NK = 808$ equality constraints (boundary conditions and the two Hermite-Simpson conditions per interval), and both $\mathbf{u}$ and $\mathbf{u}_m$ are bounded by ± 10.

After defining the optimization variable and the constraints of the problem, we shall choose an appropriate cost function. Since we desire to build a trajectory path for the double pendulum system, the best overall approach is to use a minimum effort controller. The cost is the Simpson-rule approximation of $\int_0^T \|\mathbf{u}\|^2 dt$:

$$\mathbf{J} = \sum_{k=0}^{K-1} \frac{h}{6} \left(\|\mathbf{u}_k\|^2 + 4\|\mathbf{u}_{k+\frac{1}{2}}\|^2 + \|\mathbf{u}_{k+1}\|^2 \right)$$

It is important that the midpoint controls $\mathbf{u}_{k+\frac{1}{2}}$ appear in the cost: they are free optimization variables, so if they were left out, the optimizer could place all the control effort there at no cost, and the knot controls $\bar{\mathbf{u}}_k$ (the ones kept as the nominal control) would become almost zero. We could include a weight matrix $\mathbf{R}$; however, for trajectory planning only (no control), it is unnecessary.

The initial guess for the solver is a straight line in state space from $\mathbf{x}_0$ to $\mathbf{x}_f$ with zero controls. What remains is to transfer this analysis to the cyipopt problem with the structure below.

```
class TrajOpt:
    def objective():
        return ...

    def gradient():
        return ...

    def constraints():
        return ...

    def jacobian():
        return ...

    def jacobianstructure(self):
        return ...
```

The *objective* function returns the cost function to minimize, the *gradient* function returns the gradient of the cost, the *constraints* function returns all of the problem's constraints, the *jacobian* function returns only the non-zeros of the jacobian matrix of the constraints (mandatory for so many optimization variables - it reduces runtime and offers faster convergence) and the *jacobianstructure* function is there to ensure that we take the sparse jacobian. All derivatives are computed with automatic differentiation (JAX) in double precision, and IPOPT uses a limited-memory quasi-Newton approximation of the Hessian with a tolerance of 10^{-6}. The results can be seen in the plot figures below.

The algorithm converged to the desired state after 50 iterations, reaching $4.2051e{-}06$ in the infeasibility factor and $8.65e{+}00$ in the objective. Contrary to a cost that ignores the midpoint controls, the optimal cost is clearly above zero, since the swing-up requires real control effort, which stays within the actuator limits.

```
Iter #49 | Obj: 8.65 | Infeasibility: 1.1863e-06
  49  8.6500131e+00 1.19e-06 4.62e-04 -11.0 5.38e-03    -  1.00e+00 1.00e+00h  1
Iter #50 | Obj: 8.65 | Infeasibility: 7.9957e-08
SUCCESS: Optimal Solution Found!
iter    objective    inf_pr   inf_du lg(mu)  ||d||  lg(rg) alpha_du alpha_pr  ls
  50  8.6500125e+00 8.00e-08 3.30e-05 -11.0 1.32e-03    -  1.00e+00 1.00e+00h  1

Number of Iterations....: 50

                                   (scaled)                 (unscaled)
Objective...............:   8.6500124793335402e+00    8.6500124793335402e+00
Dual infeasibility......:   3.3034068270501593e-05    3.3034068270501593e-05
Constraint violation....:   7.9957154092310003e-08    7.9957154092310003e-08
Complementarity.........:   9.9999999999999994e-12    9.9999999999999994e-12
Overall NLP error.......:   3.3034068270501593e-05    3.3034068270501593e-05


Number of objective function evaluations             = 86
Number of objective gradient evaluations             = 51
Number of equality constraint evaluations            = 86
Number of inequality constraint evaluations          = 0
Number of equality constraint Jacobian evaluations   = 51
Number of inequality constraint Jacobian evaluations = 0
Number of Lagrangian Hessian evaluations             = 0
Total CPU secs in IPOPT (w/o function evaluations)   =     36.921
Total CPU secs in NLP function evaluations           =    319.207

EXIT: Optimal Solution Found.
```

Fig. 3. Cyipopt converged

IV. Trajectory Tracking with TVLQR

In this section, we focus on controlling the pendulum system via Time-Varying LQR (TVLQR) using the optimal trajectory computed earlier. TVLQR is designed to track moving trajectories that require linearization at every timestep of the simulation. The first step is to linearize the dynamics around the optimal states and controls, deriving the matrices below.

$$\boldsymbol{A}_k = \left.\frac{\partial f_{\mathrm{d}}}{\partial \boldsymbol{x}}\right|_{\bar{\boldsymbol{x}}_k,\,\bar{\boldsymbol{u}}_k} \qquad \boldsymbol{B}_k = \left.\frac{\partial f_{\mathrm{d}}}{\partial \boldsymbol{u}}\right|_{\bar{\boldsymbol{x}}_k,\,\bar{\boldsymbol{u}}_k}$$

where f_d stands for the discretized dynamics function of the system through Runge-Kutta 4th Order. The optimal states and controls $\bar{\mathbf{x}}, \bar{\mathbf{u}}$ are computed from the trajectory. We choose the weight matrices

$$\mathbf{Q} = \mathrm{diag}(100,\, 100,\, 10,\, 10), \quad \mathbf{R} = \mathrm{diag}(0.01,\, 0.01),$$

which weigh both angles equally (a strongly unbalanced choice makes the controller much weaker against disturbances on one of the joints). The terminal cost is set to $\mathbf{Q}_N = \mathbf{P}_\infty$, the cost-to-go of the infinite-horizon LQR at the top equilibrium, so that the end of the trajectory hands over smoothly to the controller that holds the pendulum on top. We then solve the two equations from the TVLQR algorithm as seen below.

$$\begin{aligned}\mathbf{K}_k &= (\mathbf{R}_k + \mathbf{B}_k^T\mathbf{P}_{k+1}\mathbf{B}_k)^{-1}\mathbf{B}_k^T\mathbf{P}_{k+1}\mathbf{A}_k \\ \mathbf{P}_k &= \mathbf{Q}_k + \mathbf{A}_k^T\mathbf{P}_{k+1}(\mathbf{A}_k - \mathbf{B}_k\mathbf{K}_k)\end{aligned}$$

starting from $\mathbf{P}_K = \mathbf{Q}_N$. We extract only the gains $\mathbf{K}_k$, needed for the controller later on. This algorithm solves the regular LQR control with the only difference being that the goal is to follow a moving path, not a static state. After formulating the core path of this section, the feedback controller remains to be defined. For a more realistic analysis, Gaussian noise is added to the measurement, and the commanded torque is limited by the actuator bounds, meaning that the controller will be of the form:

$$\mathbf{u}_k = \mathrm{sat}\left(\bar{\mathbf{u}}_k - \mathbf{K}_k(\mathbf{x}_{measured} - \bar{\mathbf{x}}_k)\right)$$

where $\bar{\mathbf{u}}_k, \bar{\mathbf{x}}_k$ are the nominal controls and states, $\mathbf{x}_{measured}$ is the current state plus Gaussian noise, and $\mathrm{sat}(\cdot)$ clips each torque to $[-10, 10]$ N m. This feedback law ensures the pendulum follows the trajectory under the influence of noise. Of course, how well this theory is verified strongly depends on the level of noise.

V. Stabilization on Top and Disturbance Rejection

The trajectory runs for a total time of 5 seconds. After 5 seconds, the TVLQR controller "shuts down" since it has nothing to follow. To this end, we switch to an infinite horizon LQR controller to stabilize the double pendulum system at the top equilibrium point $\mathbf{x}_f$. The dynamics are linearized around $(\mathbf{x}_f, \mathbf{0})$, giving $\mathbf{A}, \mathbf{B}$ (the upright equilibrium is open-loop unstable), and with $\mathbf{e} = \mathbf{x} - \mathbf{x}_f$ the control law is $\mathbf{u} = -\mathbf{K}_\infty\mathbf{e}$, where

$$\begin{aligned}\mathbf{P}_\infty &= \mathbf{Q} + \mathbf{A}^T\mathbf{P}_\infty\mathbf{A} - \mathbf{A}^T\mathbf{P}_\infty\mathbf{B}(\mathbf{R} + \mathbf{B}^T\mathbf{P}_\infty\mathbf{B})^{-1}\mathbf{B}^T\mathbf{P}_\infty\mathbf{A} \\ \mathbf{K}_\infty &= (\mathbf{R} + \mathbf{B}^T\mathbf{P}_\infty\mathbf{B})^{-1}\mathbf{B}^T\mathbf{P}_\infty\mathbf{A}\end{aligned}$$

is the solution of the discrete algebraic Riccati equation. Since this gain minimizes the cost over an infinite time window, it is the appropriate choice for holding a fixed equilibrium.

A. Disturbance rejection

A state-feedback law such as $\mathbf{K}_\infty$ brings the pendulum back to the top after a short push, but it cannot reject a force that stays on: a constant disturbance torque $\mathbf{d}$ moves the closed-loop equilibrium away from $\mathbf{x}_f$, and the pendulum settles with a permanent offset. To remove it, we add integral action (LQI). The two joint-angle errors are integrated,

$$\mathbf{z}_{k+1} = \mathbf{z}_k + h\,\mathbf{C}\mathbf{e}_k, \quad \mathbf{C} = \begin{bmatrix}\mathbf{I}_2 & \mathbf{0}\end{bmatrix},$$

and the augmented system $\boldsymbol{\xi} = [\mathbf{e}^T\ \mathbf{z}^T]^T$ with

$$\mathbf{A}_a = \begin{bmatrix}\mathbf{A} & \mathbf{0} \\ h\mathbf{C} & \mathbf{I}_2\end{bmatrix}, \quad \mathbf{B}_a = \begin{bmatrix}\mathbf{B} \\ \mathbf{0}\end{bmatrix}, \quad \mathbf{Q}_a = \begin{bmatrix}\mathbf{Q} & \mathbf{0} \\ \mathbf{0} & q_I\mathbf{I}_2\end{bmatrix}$$

is controlled with the infinite-horizon LQR gain $\mathbf{K}_a$ computed for $(\mathbf{A}_a, \mathbf{B}_a, \mathbf{Q}_a, \mathbf{R})$, i.e. $\mathbf{u} = -\mathbf{K}_a\boldsymbol{\xi}$. We use $q_I = 200$: a larger value removes offsets faster but makes the controller more sensitive to measurement noise. To avoid integrator wind-up, $\mathbf{z}$ is only updated while the commanded torque is within the actuator limits. The integrator starts at zero when the controller switches from TVLQR to the hold controller.

B. Simulation setup

Disturbances are modeled as extra torques $\mathbf{d}_k$ applied to the joints of the plant (a force $\mathbf{F}$ at the tip is equivalent to the torque $\mathbf{J}^T\mathbf{F}$), so that

$$\mathbf{x}_{k+1} = f_d(\mathbf{x}_k,\ \mathrm{sat}(\mathbf{u}_k) + \mathbf{d}_k)\,,$$

whereas measurement noise only affects the state fed to the controller. The disturbance scenario is as follows: the pendulum swings up and is held on top (0–5 s), receives a $0.1\,s$ push of $+10$ N m on joint 1 at $t = 7\,s$, a $0.1\,s$ push of -6 N m on joint 2 at $t = 9.5\,s$, and receives from $t = 11\,s$ a constant 2 N m on joint 1 that never goes away. We compare the controller with integral action to the same pipeline using the plain $\mathbf{K}_\infty$ in the hold phase.

VI. Results

A. Measurement noise

Figure **??** shows the closed loop for Gaussian measurement noise with $\sigma = 0.001,\ 0.01$ and 0.05, with the torque limit active. For $\sigma = 0.001$, the noise is almost unnoticeable. As σ grows, the tracking error and the torque activity increase, but the pendulum remains stabilized on top. A noise level of $\sigma = 0.5$, as used in an earlier version of this work, requires torques far above the 10 N m actuator limit; with the limit active the pendulum no longer stays on top, so such levels are not considered here. We also notice that the controller cannot track the trajectory with perfect precision, for various reasons. The double pendulum system is a highly nonlinear system that we linearize using Taylor Series, ignoring the high-order terms of the expansion, so it is natural for the controller path to diverge when the pendulum moves fast or swings wide. In addition, the collocation trajectory assumes interpolated controls, while the simulation holds the control constant during each time step.

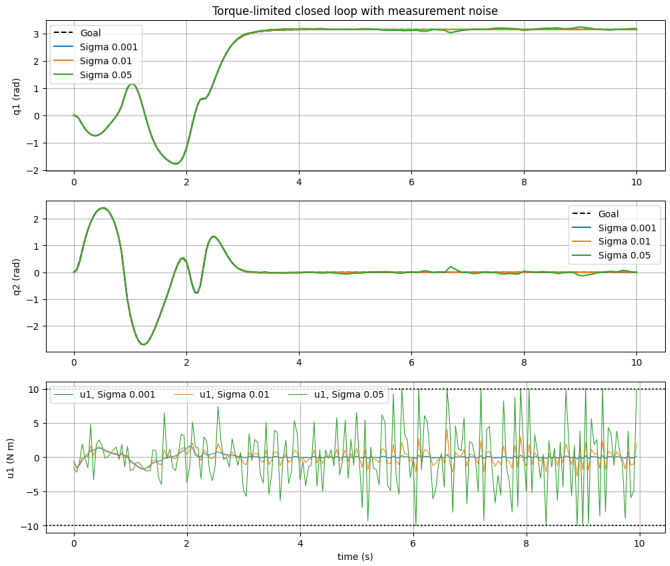


Fig. 4. Enter Caption

B. Disturbance rejection

Both controllers recover from the two short pushes. Under the sustained 2 N m force, the plain $\mathbf{K}_\infty$ settles with a steady offset of approximately 0.15 rad in q_1 and 0.28 rad in q_2, whereas the controller with integral action returns to the upright position with practically zero error. The price of the integrator is a slightly larger and slower transient after a sharp push. An animation of the scenario is available in the accompanying repository.

VII. Conclusion

A swing-up trajectory of a double pendulum was computed with Hermite-Simpson collocation and IPOPT, tracked with TVLQR, and the pendulum was stabilized on top with an infinite-horizon LQR with integral action. The integral action makes the difference between a controller that only recovers from short pushes and one that also rejects sustained forces. The results are subject to the following limitations. The controller is designed and simulated with the same model, so model mismatch, friction and actuator delay are not covered. Disturbance rejection is limited by the actuator bounds, so a large enough or long enough disturbance will still make the pendulum fall, and the results here are empirical rather than guaranteed. Finally, the angle errors are not wrapped, so after a very large push the controller may unwind several full turns before returning to the upright position.